

K-Means Clustering

Importing the libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset = pd.read_csv('Iris.csv')

dataset.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---
0    Id                   150 non-null    int64
1    SepalLengthCm        150 non-null    float64
2    SepalWidthCm         150 non-null    float64
3    PetalLengthCm        150 non-null    float64
4    PetalWidthCm         150 non-null    float64
5    Species              150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB

dataset.shape

(150, 6)

X = dataset.iloc[:,[3,4]].values

print(X)

[[1.4 0.2]
 [1.4 0.2]
 [1.3 0.2]
 [1.5 0.2]
 [1.4 0.2]
 [1.7 0.4]
 [1.4 0.3]
 [1.5 0.2]
 [1.4 0.2]
 [1.5 0.1]
 [1.5 0.2]
 [1.6 0.2]
 [1.4 0.1]
 [1.1 0.1]
 [1.2 0.2]]
```

[1.5 0.4]
[1.3 0.4]
[1.4 0.3]
[1.7 0.3]
[1.5 0.3]
[1.7 0.2]
[1.5 0.4]
[1. 0.2]
[1.7 0.5]
[1.9 0.2]
[1.6 0.2]
[1.6 0.4]
[1.5 0.2]
[1.4 0.2]
[1.6 0.2]
[1.6 0.2]
[1.5 0.4]
[1.5 0.1]
[1.4 0.2]
[1.5 0.1]
[1.2 0.2]
[1.3 0.2]
[1.5 0.1]
[1.3 0.2]
[1.5 0.2]
[1.3 0.3]
[1.3 0.3]
[1.3 0.2]
[1.6 0.6]
[1.9 0.4]
[1.4 0.3]
[1.6 0.2]
[1.4 0.2]
[1.5 0.2]
[1.4 0.2]
[4.7 1.4]
[4.5 1.5]
[4.9 1.5]
[4. 1.3]
[4.6 1.5]
[4.5 1.3]
[4.7 1.6]
[3.3 1.]
[4.6 1.3]
[3.9 1.4]
[3.5 1.]
[4.2 1.5]
[4. 1.]
[4.7 1.4]

[3.6 1.3]
[4.4 1.4]
[4.5 1.5]
[4.1 1.]
[4.5 1.5]
[3.9 1.1]
[4.8 1.8]
[4. 1.3]
[4.9 1.5]
[4.7 1.2]
[4.3 1.3]
[4.4 1.4]
[4.8 1.4]
[5. 1.7]
[4.5 1.5]
[3.5 1.]
[3.8 1.1]
[3.7 1.]
[3.9 1.2]
[5.1 1.6]
[4.5 1.5]
[4.5 1.6]
[4.7 1.5]
[4.4 1.3]
[4.1 1.3]
[4. 1.3]
[4.4 1.2]
[4.6 1.4]
[4. 1.2]
[3.3 1.]
[4.2 1.3]
[4.2 1.2]
[4.2 1.3]
[4.3 1.3]
[3. 1.1]
[4.1 1.3]
[6. 2.5]
[5.1 1.9]
[5.9 2.1]
[5.6 1.8]
[5.8 2.2]
[6.6 2.1]
[4.5 1.7]
[6.3 1.8]
[5.8 1.8]
[6.1 2.5]
[5.1 2.]
[5.3 1.9]
[5.5 2.1]

```
[5.  2. ]
[5.1 2.4]
[5.3 2.3]
[5.5 1.8]
[6.7 2.2]
[6.9 2.3]
[5.  1.5]
[5.7 2.3]
[4.9 2. ]
[6.7 2. ]
[4.9 1.8]
[5.7 2.1]
[6.  1.8]
[4.8 1.8]
[4.9 1.8]
[5.6 2.1]
[5.8 1.6]
[6.1 1.9]
[6.4 2. ]
[5.6 2.2]
[5.1 1.5]
[5.6 1.4]
[6.1 2.3]
[5.6 2.4]
[5.5 1.8]
[4.8 1.8]
[5.4 2.1]
[5.6 2.4]
[5.1 2.3]
[5.1 1.9]
[5.9 2.3]
[5.7 2.5]
[5.2 2.3]
[5.  1.9]
[5.2 2. ]
[5.4 2.3]
[5.1 1.8]]
```

```
from sklearn.cluster import KMeans
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters= i, init='k-means++',random_state= 42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11),wcss)
plt.title("Elbow method")
plt.xlabel("No. of clusters")
plt.ylabel("WCSS")
plt.show()
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

```
warnings.warn(
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

```
warnings.warn(
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

```
warnings.warn(
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

```
warnings.warn(
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

```
warnings.warn(
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

```
warnings.warn(
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

```
warnings.warn(
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

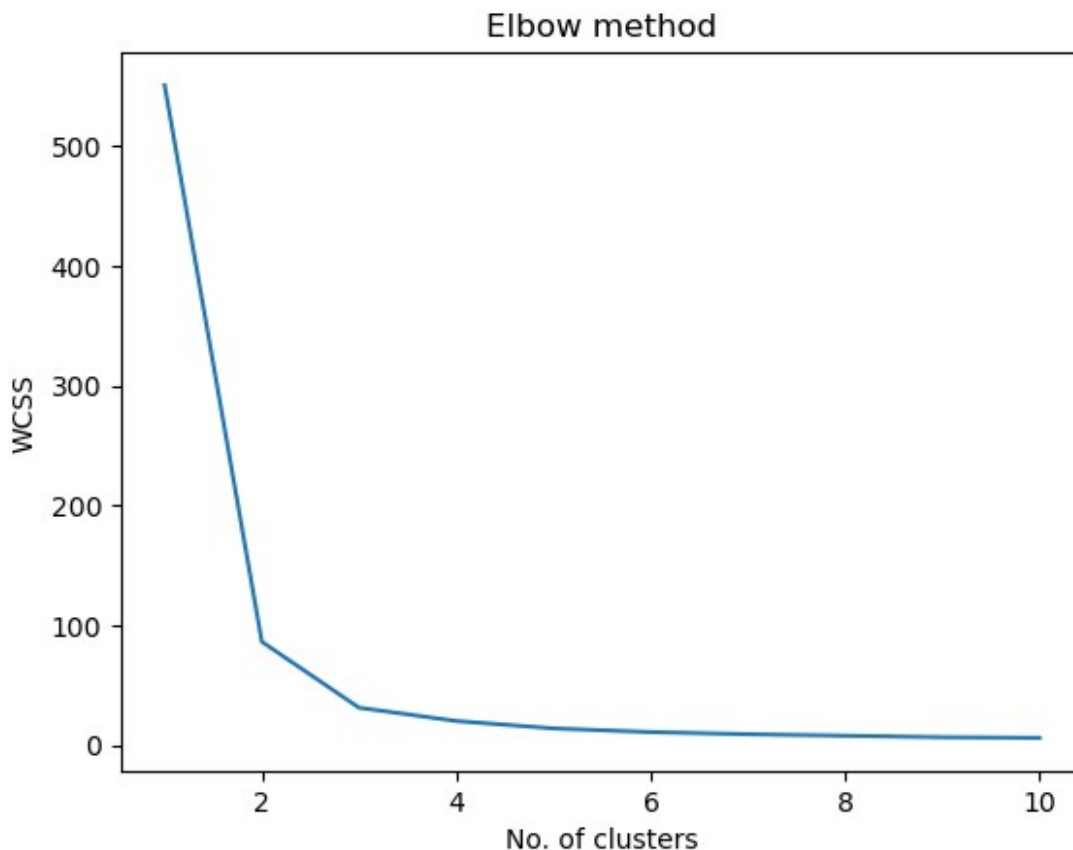
```
warnings.warn(
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
```

a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable `OMP_NUM_THREADS=1`.

```
warnings.warn(  
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\  
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have  
a memory leak on Windows with MKL, when there are less chunks than  
available threads. You can avoid it by setting the environment  
variable OMP_NUM_THREADS=1.  
warnings.warn(  

```



```
kmeans = KMeans(n_clusters= 5, init='k-means++',random_state= 42)  
kmeans.fit(X)
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\  
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have  
a memory leak on Windows with MKL, when there are less chunks than  
available threads. You can avoid it by setting the environment  
variable OMP_NUM_THREADS=1.  
warnings.warn(  

```

```
KMeans(n_clusters=5, random_state=42)
```

```
# The lowest SSE value
```

```
kmeans.inertia_
```

```
14.147293864755936
```

```
# Final locations of the centroid
```

```
kmeans.cluster_centers_
```

```
array([[3.884      , 1.192      ],
       [1.464      , 0.244      ],
       [4.71142857, 1.54571429],
       [6.35454545, 2.12727273],
       [5.47241379, 2.07931034]])
```

```
kmeans.n_iter_
```

```
10
```

```
kmeans.labels_
```

```
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
      1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
      1, 1, 1, 1, 1, 1, 2, 2, 2, 0, 2, 2, 2, 0, 2, 0, 0, 0, 0, 2, 0,
2,
      2, 0, 2, 0, 2, 0, 2, 2, 0, 2, 2, 2, 2, 0, 0, 0, 0, 2, 2, 2, 2,
2,
      0, 0, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 3, 4, 4, 4, 4, 3, 2, 3, 4,
3,
      4, 4, 4, 4, 4, 4, 4, 3, 3, 2, 4, 2, 3, 2, 4, 3, 2, 2, 4, 4, 3,
3,
      4, 2, 4, 3, 4, 4, 2, 4, 4, 4, 4, 4, 4, 4, 2, 4, 4, 2]])
```

```
y_kmeans = kmeans.fit_predict(X)
```

```
C:\Users\LENOVO\AppData\Roaming\Python\Python312\site-packages\
sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have
a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment
variable OMP_NUM_THREADS=1.
```

```
warnings.warn(
```

```
print(y_kmeans)
```

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1
  1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 0 2 2 2 0 2 0 0 0 0 2 0 2 2 0 2 0 2 0
2 2
  0 2 2 2 2 0 0 0 0 2 2 2 2 2 0 0 2 2 0 0 0 0 0 0 0 0 3 4 4 4 4 3 2 3 4
3 4
  4 4 4 4 4 4 3 3 2 4 2 3 2 4 3 2 2 4 4 3 3 4 2 4 3 4 4 2 4 4 4 4 4 4 4
4 4]
```

```

2 4
4 2]

plt.scatter(X[y_kmeans == 0,0],X[y_kmeans == 0,1],s=100, c = 'red',
label = "Cluster 1")
plt.scatter(X[y_kmeans == 1,0],X[y_kmeans == 1,1],s=100, c = 'blue',
label = "Cluster 2")
plt.scatter(X[y_kmeans == 2,0],X[y_kmeans == 2,1],s=100, c =
'lightgreen', label = "Cluster 3")
plt.scatter(X[y_kmeans == 3,0],X[y_kmeans == 3,1],s=100, c = 'black',
label = "Cluster 4")
plt.scatter(X[y_kmeans == 4,0],X[y_kmeans == 4,1],s=100, c =
'magenta', label = "Cluster 5")
plt.scatter(kmeans.cluster_centers_[ :,0],kmeans.cluster_centers_[ :,1],
s = 300, c = 'Yellow',label = 'Centroids')
plt.title("Clusters of Customers",size = 25)
plt.xlabel("Annual Income {k$}")
plt.ylabel("Spending Scores{1 to 100}")
plt.legend()
plt.show()

```

